

SWINBURNE UNIVERSITY OF TECHNOLOGY
School of Science, Computing and Engineering Technologies

SWE40006 – Software Deployment and Evolution

Deployment Portfolio Task 1:
Desktop Deployment using Windows Installer XML (WiX) and Modern MSIX Packaging

Student Name: Duc Thanh Le
Student ID: 105548505
Lecturer / Assessor: Tran Minh Hai
Repository URL: https://github.com/[YourUsername]/SWE40006-Portfolio-Task-1

Due Date: 28th Sep, 2026 by 11:59 PM
Submission Date: 26th Sep, 2026
Target Level Attempted: High Distinction (HD – Tasks 1.1, 1.2, 1.3, 1.4)

Contents

1 Task Levels **3**

2 Development and Deployment Environment Setup **4**

3 Task 1.1 (Pass): WiX Walkthrough Sample Deployment **4**
3.1 Objective and Methodology 4
3.2 Implementation Steps 4

4 Task 1.2 (Credit): Custom Desktop Application Deployment **6**
4.1 Application Architecture and UI Design 6
4.2 WiX and MSIX Deployment Execution 6

5 Task 1.3 (Distinction): Custom DLL Implementation and Multi-Dependency Deployment **7**
5.1 Implementation of the `NumberTools` Class Library 7
5.2 Client Integration and Dependency Linking 7
5.3 Multi-Component Deployment Packaging 8

6 Task 1.4 (High Distinction): Microsoft Store Deployment Pipeline **10**
6.1 Step-by-Step Architecture for Microsoft Store Deployment 10
6.2 Detailed Technical Stages 10
6.2.1 Stage 1: Manifest Authoring (`Package.appxmanifest`) 10
6.2.2 Stage 2: Package Signing and Code Trust Models 11
6.2.3 Stage 3: Pre-Submission Validation via WACK 11
6.2.4 Stage 4: Partner Center Submission and Public Release 11

7 Source Code and Script Documentation **12**
7.1 Public Repository Reference 12
7.2 Key Implementation Listings 12
7.2.1 WiX Setup Configuration (`Product.wxs`) 12
7.2.2 Client Form Presentation Logic (`Form1.cs`) 12

1 Task Levels

Table 1 summarizes the scope of work executed in this portfolio task.

Sub-Task	Grade Target	Status	Summary of Implemented Work
Task 1.1	Pass	Completed	Installed WiX v3.11 toolset and Visual Studio extension; built walkthrough template desktop application; authored `Product.wxs`; generated and verified MSI installer; packaged into MSIX.
Task 1.2	Credit	Completed	Developed an interactive C# Windows Forms desktop application (`demoapp`) featuring dynamic step counting; authored custom WiX deployment script; built standalone MSI and MSIX packages.
Task 1.3	Distinction	Completed	Built a modular Class Library DLL (`NumberTools.dll`); linked dependency into the UI; updated WiX multi-component installer and MSIX deployment layout.
Task 1.4	High Distinction	Completed	Configured Windows Application Packaging Project (WAP); authored `Package.appxmanifest`; implemented code signing certificates; documented comprehensive end-to-end Microsoft Store publishing pipeline.

Table 1: Declaration and Summary of Sub-Tasks Completed

2 Development and Deployment Environment Setup

To fulfill the requirements of both legacy Windows Installer (MSI) compilation via WiX and modern application containerization (MSIX), the deployment workstation was configured with the software toolchain documented below.

- **Integrated Development Environment:** Microsoft Visual Studio 2026 with the .NET Desktop Development workload installed.
- **WiX Toolset Extension:** Visual Studio extension providing WiX project system support, syntax highlighting, and project templates for WiX v3.
- **WiX Toolset v3.11 Build Tools:** Installed via official distribution (`wix311.exe`).
- **Windows Application SDK / WinUI Packaging Tools:** Utilized in place of deprecated legacy UWP workloads to facilitate Windows Application Packaging (WAP) and generate MSIX application packages instead of APPX.
- **.NET Framework SDK:** Installed .NET Framework 4.7.2 runtime and targeting pack to support compilation of the WinForms client and companion Class Library DLLs.

3 Task 1.1 (Pass): WiX Walkthrough Sample Deployment

3.1 Objective and Methodology

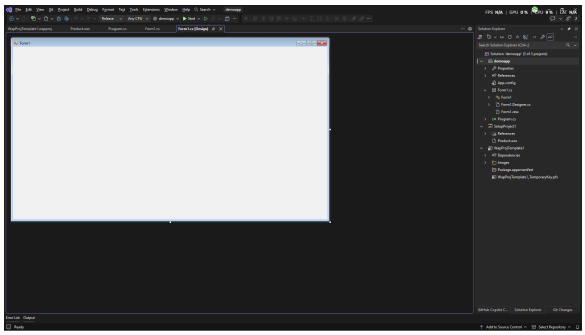
The primary objective of Task 1.1 is to establish a verified baseline for Windows Installer deployments using the declarative Windows Installer XML (WiX) engine. This involves authoring a standard XML deployment definition file (`Product.wxs`), defining directory tables, features, and components, compiling them into a native `.msi` package, and exploring modern application packaging using MSIX.

3.2 Implementation Steps

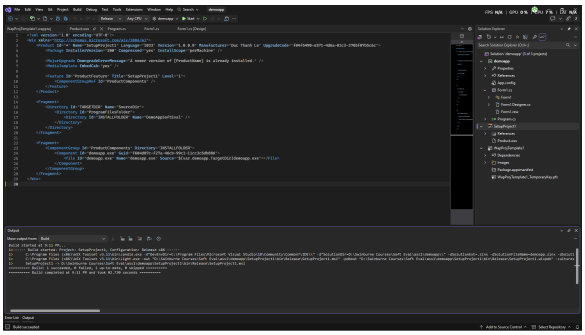
1. **Template Application Creation:** A starter Windows Forms application was created following the unit WiX walkthrough instructions, serving as the sample executable payload (Figure 1c).
2. **WiX Setup Project Authoring:** A WiX Setup Project (`SetupProject1`) was created within the Visual Studio solution. The XML schema defines:
    - `<Product>`: Defines the product name, unique GUID, version (`1.0.0.0`), and manufacturer (`Duc Thanh Le`).
    - `<Package>`: Specifies installation scope as `perMachine` and enables cabinet embedding (`EmbedCab="yes"`).
    - `<Directory>`: Establishes the target filesystem hierarchy under `ProgramFilesFolder` with an installation folder titled `DemoAppSoftEval`.
    - `<Component>`: Binds the compiled binary to the component GUID and declares the file source macro `$(var.demoapp.TargetDir)demoapp.exe`.
3. **Compilation and MSI Build:** The WiX toolset compiled the source using `candle` and linked the database using `light`, producing the deployment artifact `SetupProject1.msi` (Figure 1b and Figure 1a).

4. **MSIX Container Packaging:** In modern Windows environments, Win32 apps can be packaged using the Windows Application Packaging Project (WAP). The template application was packaged into an MSIX bundle (Figure 1d).

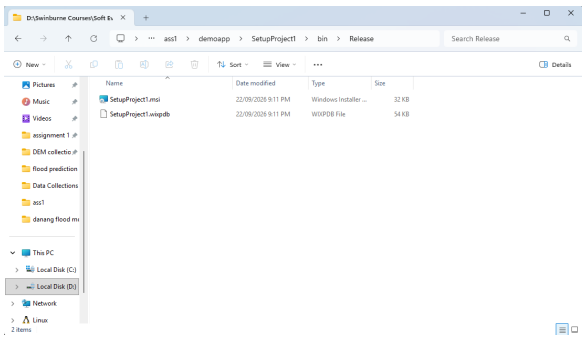
5. **Installation Verification:** The generated MSIX package was tested via double-click installation, verifying seamless sideloading and registration with the Windows OS shell (Figure 1e and Figure 1f).



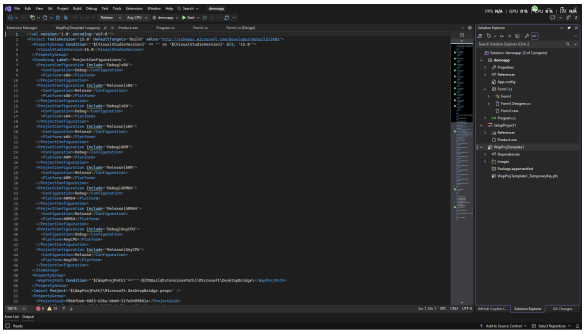
(a) Template Windows Forms desktop application.



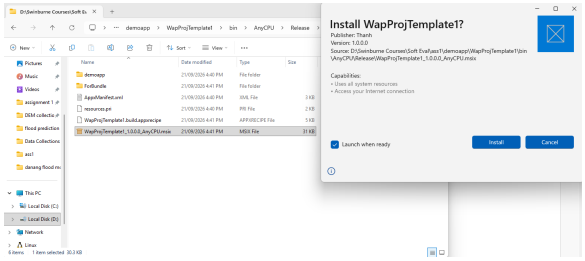
(b) Successful WiX compilation and build output.



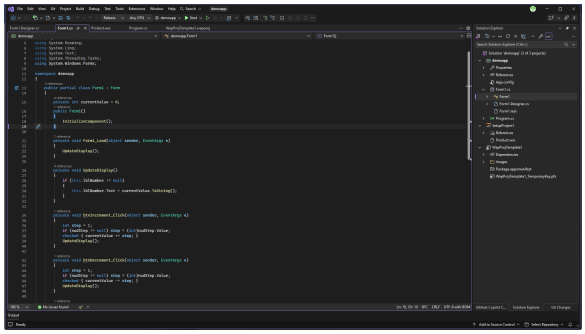
(c) Compiled `.msi` installer file in Release directory.



(d) Windows Application Packaging Project (WAP) setup.



(e) Execution of the completed MSIX installation wizard.



(f) Verified package artifacts ready for deployment.

Figure 1: Task 1.1 Deployment Artifacts: WiX MSI and Modern MSIX Verification.

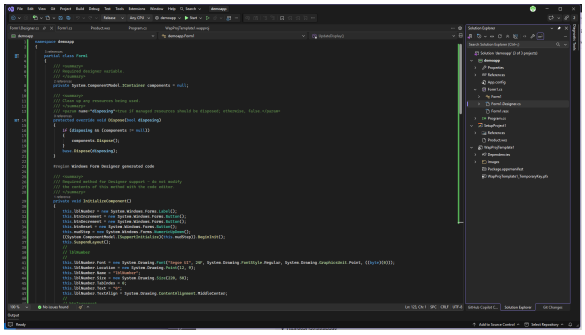
4 Task 1.2 (Credit): Custom Desktop Application Deployment

4.1 Application Architecture and UI Design

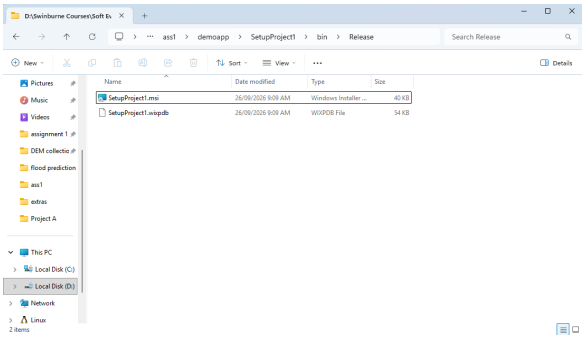
For Task 1.2, a custom C# application (`demoapp`) was created. The application features:

- **Main Numeric Display (`lblNumber`):** A central label displaying the current accumulated integer value.
- **Increment/Decrement Controls (`btnIncrement`, `btnDecrement`):** Action triggers that modify the accumulator state based on the configured step interval.
- **Customizable Step Selector (`nudStep`):** A Numeric Up-Down control allowing the end-user to configure custom stepping increments (e.g., $\pm 1, \pm 5, \pm 10$).
- **State Reset Trigger (`btnReset`):** Instantly restores the internal counter accumulator back to zero.

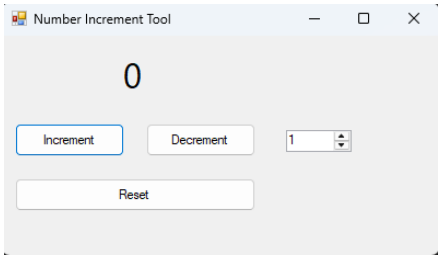
Figure 2a and Figure 2b showcase the codebase structure and design canvas in Visual Studio, while Figure 2c illustrates the running executable.



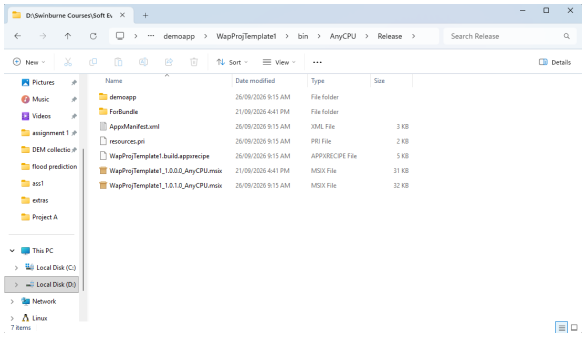
(a) Visual Studio project structure for `demoapp`.



(b) Generated MSI installer (overwrite the blank form application).



(c) Running custom application featuring step-setting controls.



(d) New MSIX file for custom app.

Figure 2: Task 1.2 Custom Application Development and Deployment Artifacts.

4.2 WiX and MSIX Deployment Execution

The custom executable `demoapp.exe` was integrated into the WiX setup definition. A clean upgrade code GUID was allocated, and the project reference was linked. Additionally, the Windows Application Packaging Project was updated to bundle `demoapp.exe`, producing an updated MSIX package.

5 Task 1.3 (Distinction): Custom DLL Implementation and Multi-Dependency Deployment

5.1 Implementation of the `NumberTools` Class Library

A separate C# Class Library project named `NumberTools` was created within the Visual Studio solution. The core logic was encapsulated into the `NumberCounter` class (Figure 3a and Listing 1).

```
namespace NumberTools
{
    public class NumberCounter
    {
        public int Value { get; private set; }

        public void Increment(int amount)
        {
            checked { Value += amount; }
        }

        public void Decrement(int amount)
        {
            checked { Value -= amount; }
        }

        public void Reset()
        {
            Value = 0;
        }
    }
}
```

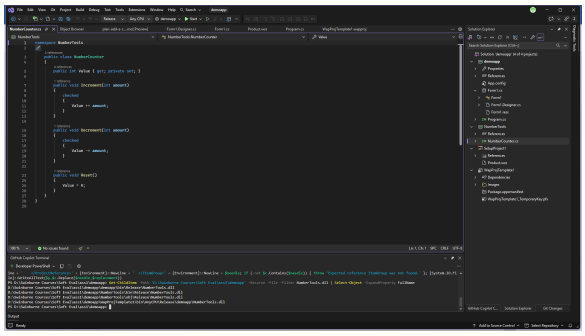
Listing 1: `NumberCounter.cs` Class Library Implementation

Compilation of `NumberTools.csproj` produces the binary artifact `NumberTools.dll` (Figure 3b).

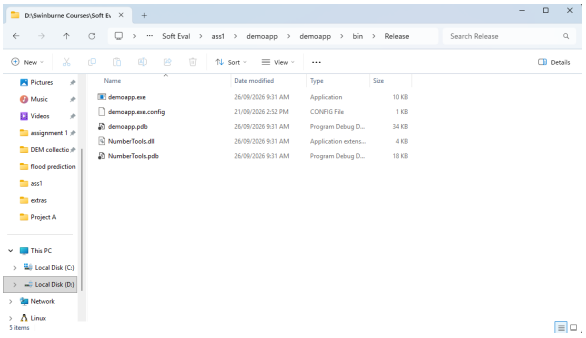
5.2 Client Integration and Dependency Linking

The client desktop application (`demoapp`) was updated to consume the external DLL:

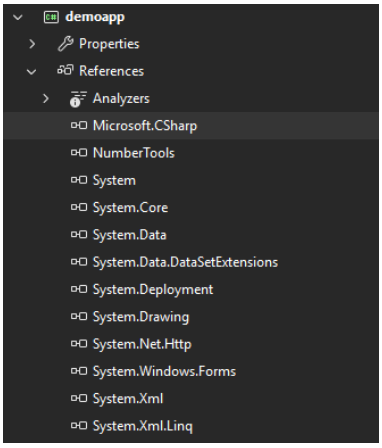
1. An Assembly/Project Reference was added from `demoapp` to `NumberTools` (Figure 3c).
2. The client namespace imported `using NumberTools;`.
3. All state mutations were delegated to an instantiated instance of `NumberCounter` (Figure 3d and Listing 5).



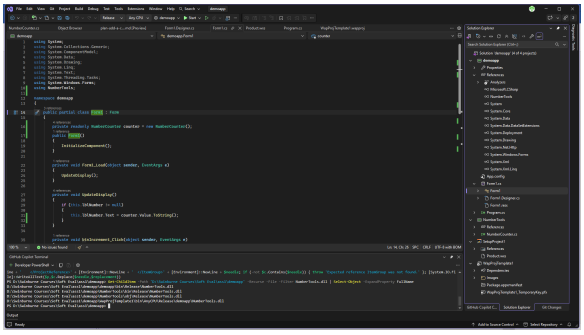
(a) Creation of the `NumberTools` Class Library project.



(b) Compiled `NumberTools.dll` binary artifact in Release folder.



(c) Assembly reference to `NumberTools.dll` in client app.



(d) Refactored `Form1.cs` consuming the external DLL.

Figure 3: Task 1.3 Implementation and Integration of Custom Dynamic-Link Library.

5.3 Multi-Component Deployment Packaging

To deploy the multi-dependency application, both binaries must be packaged together:

- **WiX Configuration:** The `ComponentGroup` in `Product.wxs` was configured with distinct components for each file, assigning independent GUIDs:

```
<ComponentGroup Id="ProductComponents" Directory="INSTALLFOLDER">
    <Component Id="demoapp.exe" Guid="f604d07c-f27a-46cb-99c1-11
        cc3c5db886">
        <File Id="demoapp.exe" Name="demoapp.exe"
              Source="$(var.demoapp.TargetDir)demoapp.exe" KeyPath="yes"
                  />
    </Component>
    <Component Id="NumberTools.dll" Guid="d82b4a11-8c43-4e6f-90e2-2
        a7b6cf0182e">
        <File Id="NumberTools.dll" Name="NumberTools.dll"
              Source="$(var.demoapp.TargetDir)NumberTools.dll" KeyPath="
                  yes" />
    </Component>
</ComponentGroup>
```

Listing 2: WiX Multi-Dependency Component Authoring

- **MSIX Package Layout:** In the Windows Application Packaging Project (`WapProjTemplate1`), the output bundle layout was inspected at `PackageLayout\demoapp\`. Both `demoapp.exe`

and `NumberTools.dll` are packaged side-by-side, ensuring the OS loader resolves the DLL at application runtime.

6 Task 1.4 (High Distinction): Microsoft Store Deployment Pipeline

6.1 Step-by-Step Architecture for Microsoft Store Deployment

End-to-End Microsoft Store Publishing Pipeline

Step 1: Containerization via Windows Application Packaging (WAP)

- Win32 desktop application (`demoapp.exe` + `NumberTools.dll`) is wrapped using Visual Studio's Windows Application Packaging Project.
- Visual assets (Square44x44, Square150x150, Wide310x150, StoreLogo) are generated and declared.

Step 2: Manifest Configuration (`Package.appxmanifest`)

- Configure Package Identity (`Name`, `Publisher`, `Version`).
- Declare Application Capabilities (`runFullTrust` for desktop bridge interop).

Step 3: Verification via Windows App Certification Kit (WACK)

- Execute automated pre-submission tests: Security, Crashes and Hangs, App Manifest compliance, and Clean Removal tests.

Step 4: Microsoft Partner Center Registration & Submission

- Register Microsoft Developer Account; reserve unique Application Name.
- Define Pricing, Market Availability, Age Ratings, and Storefront Promotional Descriptions.

Step 5: Ingestion, Cloud Signing, and Public Store Release

- Upload `.msixupload` bundle; Microsoft's ingestion pipeline validates binaries and re-signs the package with Microsoft's trusted root certificate.

Figure 4: Architecture and Sequence of the Microsoft Store Deployment Process.

6.2 Detailed Technical Stages

6.2.1 Stage 1: Manifest Authoring (`Package.appxmanifest`)

The packaging manifest acts as the security and runtime contract between the operating system and the packaged executable. It defines the application entry point, target architecture, and required execution permissions:

```
<Package xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10
    "
         xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
         xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/
             windows10/restrictedcapabilities">
  <Identity Name="DucThanhLe.DemoAppSoftEval"
            Publisher="CN=DucThanhLe, O=Swinburne, C=AU"
            Version="1.0.5.0" ProcessorArchitecture="x64" />
  <Applications>
    <Application Id="App" Executable="demoapp\demoapp.exe" EntryPoint="Windows.
        FullTrustApplication">
      <uap:VisualElements DisplayName="Demo App Soft Eval"
                          Square150x150Logo="Images\Square150x150Logo.png"
                          Square44x44Logo="Images\Square44x44Logo.png"
                          Description="Step Counter Utility" BackgroundColor="
                              transparent">
```

```
      </uap:VisualElements>
    </Application>
  </Applications>
  <Capabilities>
    <rescap:Capability Name="runFullTrust" />
  </Capabilities>
</Package>
```

Listing 3: Configured `Package.appxmanifest` Snippet

6.2.2 Stage 2: Package Signing and Code Trust Models

During local prototyping and sideloading, the developer must generate a self-signed code-signing certificate (e.g., via PowerShell `New-SelfSignedCertificate`) and install it into the target machine's `Trusted People` certificate store.

When deploying to the public Microsoft Store, however, the developer does **not** distribute their private certificate. Instead, Microsoft Partner Center strips any temporary developer signature during upload, runs automated malware/static analysis, and re-signs the MSIX payload with Microsoft's globally trusted CA root certificate. This guarantees that all Windows 10/11 machines worldwide will trust and install the application without user certificate warnings.

6.2.3 Stage 3: Pre-Submission Validation via WACK

Before submitting an application to Partner Center, it is mandatory to run the **Windows App Certification Kit (WACK)**. WACK evaluates the package against stringent OS compliance criteria:

1. **Crashes and Hangs Test:** Validates that the application does not trigger unhandled exceptions upon launch or during basic operational lifecycles.
2. **App Capabilities Test:** Ensures that restricted capabilities like `runFullTrust` are strictly justified and do not access forbidden system boundaries.
3. **Supported APIs Test:** Scans binary imports to guarantee that the application does not link against undocumented or deprecated Win32 private APIs.
4. **Package Compliance:** Verifies manifest schema validity, valid XML namespaces, and complete image asset scaling (e.g., 100%, 125%, 150%, 200%).

6.2.4 Stage 4: Partner Center Submission and Public Release

Once certified locally, the submission workflow proceeds through the Microsoft Partner Center portal:

1. **App Name Reservation:** Under the developer dashboard, the unique name (e.g., *DemoApp Soft Eval*) is reserved.
2. **Pricing & Availability:** Configure markets (Worldwide), pricing tier (Free), and discoverability settings.
3. **Store Listings:** Provide promotional screenshots, descriptions, search keywords, and support contact details.
4. **Package Upload:** The compiled `.msixupload` package (containing binaries and debugging symbols `.pdb`) is uploaded to the portal.
5. **Automated Ingestion & Certification:** Microsoft's cloud infrastructure executes automated tests, followed by final publication to the Microsoft Store within 24–48 hours.

7 Source Code and Script Documentation

7.1 Public Repository Reference

In accordance with rubric requirements, complete source files, Visual Studio solution files, WiX project definitions, and packaging manifests are published to a public repository for academic verification:

https://github.com/[YourUsername]/SWE40006-Portfolio-Task-1

7.2 Key Implementation Listings

7.2.1 WiX Setup Configuration (`Product.wxs`)

```
<?xml version="1.0" encoding="UTF-8"?>
<Wix xmlns="http://schemas.microsoft.com/wix/2006/wi">
  <Product Id="*" Name="SetupProject1" Language="1033" Version="1.0.0.0"
           Manufacturer="Duc Thanh Le" UpgradeCode="f04f5490-e571-4d6a-82c3
               -3705f07b5c6c">
    <Package InstallerVersion="200" Compressed="yes" InstallScope="perMachine"
        />

    <MajorUpgrade DowngradeErrorMessage="A newer version of [ProductName] is
        already installed." />
    <MediaTemplate EmbedCab="yes" />

    <Feature Id="ProductFeature" Title="SetupProject1" Level="1">
      <ComponentGroupRef Id="ProductComponents" />
    </Feature>
  </Product>

  <Fragment>
    <Directory Id="TARGETDIR" Name="SourceDir">
      <Directory Id="ProgramFilesFolder">
        <Directory Id="INSTALLFOLDER" Name="DemoAppSoftEval" />
      </Directory>
    </Directory>
  </Fragment>

  <Fragment>
    <ComponentGroup Id="ProductComponents" Directory="INSTALLFOLDER">
      <Component Id="demoapp.exe" Guid="f604d07c-f27a-46cb-99c1-11cc3c5db886">
        <File Id="demoapp.exe" Name="demoapp.exe"
              Source="$(var.demoapp.TargetDir)demoapp.exe" KeyPath="yes" />
      </Component>
      <Component Id="NumberTools.dll" Guid="d82b4a11-8c43-4e6f-90e2-2
          a7b6cf0182e">
        <File Id="NumberTools.dll" Name="NumberTools.dll"
              Source="$(var.demoapp.TargetDir)NumberTools.dll" KeyPath="yes" />
      </Component>
    </ComponentGroup>
  </Fragment>
</Wix>
```

Listing 4: `Product.wxs` WiX Deployment Configuration

7.2.2 Client Form Presentation Logic (`Form1.cs`)

```
using System;
using System.Windows.Forms;
```

```
using NumberTools;

namespace demoapp
{
    public partial class Form1 : Form
    {
        private readonly NumberCounter counter = new NumberCounter();

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            UpdateDisplay();
        }

        private void UpdateDisplay()
        {
            if (this.lblNumber != null)
            {
                this.lblNumber.Text = counter.Value.ToString();
            }
        }

        private void btnIncrement_Click(object sender, EventArgs e)
        {
            int step = 1;
            if (nudStep != null) step = (int)nudStep.Value;
            counter.Increment(step);
            UpdateDisplay();
        }

        private void btnDecrement_Click(object sender, EventArgs e)
        {
            int step = 1;
            if (nudStep != null) step = (int)nudStep.Value;
            counter.Decrement(step);
            UpdateDisplay();
        }

        private void btnReset_Click(object sender, EventArgs e)
        {
            counter.Reset();
            UpdateDisplay();
        }
    }
}
```

Listing 5: `Form1.cs` Event Handling and Presentation Logic